

Week 4 — Real API: Clean Architecture + Full CRUD

Trainee Training Plan (8-Week Program) · Week 4 of 8 · TicketDesk Project · 5 working days · backend only

End-of-week deliverable: A fully working, **versioned TicketDesk Web API** built on **Clean Architecture** — Repository + Service + Dependency Injection + AutoMapper — exposing **complete CRUD** for tickets over the Week 3 database. **Goal:** the trainee builds the real backend the right way — layered, testable and pattern-based — not logic dumped in a controller.

⚡ **Note — this is an intensive week (two phases merged).** It combines the API foundation (Clean Architecture + Repository/Service/DI) and the full CRUD layer (AutoMapper + versioning) into one week. **Backend only** — React is parked for now. Each day is one ordered task that builds directly on the day before, so nobody should skip ahead.

✅ Starting point — what they should already have (end of Week 3):

- The five Domain entities as POCOs in `TicketDesk.Domain`, matching the ERD.
- `TicketDeskDbContext` in `TicketDesk.DAL` with all `DbSets` and Fluent configuration.
- EF Core **migrations** that build a real database, plus **seed data** (roles, categories) already in it.
- The solution with correct layer references (`.DAL` → `.Domain`, etc.) and each feature on its own branch + PR.

If the seeded database is not running, fix that on Day 1 before writing any API code — every endpoint this week reads from it.

Day 1 Task: Stand Up the API + Wire Clean Architecture

Learn

- The ASP.NET Core Web API project and the request pipeline (`Program.cs`, middleware, Swagger).
- The Clean Architecture dependency rule: dependencies point **inward** — `.API` → `.BL` → `.DAL` → `.Domain`.
- Registering services in the DI container (the `builder.Services` collection).

Resources

- Build your first Web API: learn.microsoft.com/aspnet/core/tutorials/first-web-api
- Dependency injection in ASP.NET Core: learn.microsoft.com/aspnet/core/fundamentals/dependency-injection
- Architectural principles (Clean Architecture): learn.microsoft.com/dotnet/architecture/modern-web-apps-azure/architectural-principles

Practice

- Confirm `TicketDesk.API` runs and Swagger opens on launch.
- Register `TicketDeskDbContext` in `Program.cs` via `AddDbContext` using the connection string.
- Add a tiny health endpoint (e.g. `GET /api/health`) to prove the pipeline works.

✅ **End of day: the API runs, Swagger opens, and the DbContext resolves through DI.**

Day 2 Task: Repository Pattern in the DAL

Learn

- Why the Repository pattern: abstract data access behind an interface, keep EF Core out of the business layer.
- A **generic** repository: `IGenericRepository<T>` with `GetAll`, `GetById`, `Add`, `Update`, `Delete`.
- Interface vs implementation, and registering both with DI (`AddScoped`).

Resources

- Persistence layer & repositories: learn.microsoft.com/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-design
- Repository pattern in C#: dotnettutorials.net/lesson/repository-design-pattern-csharp
- Generics in C#: learn.microsoft.com/dotnet/csharp/fundamentals/types/generics

Practice

- Create `IGenericRepository<T>` and `GenericRepository<T>` in `.DAL` using the `DbContext`.
- Register the repository in DI and inject it somewhere temporary to read the seeded categories.
- Confirm the repo returns real rows from the Week 3 database.

✅ **End of day: a generic repository resolves via DI and reads seeded data from the real database.**

Day 3 Task: Service Layer + DTOs + AutoMapper

Learn

- The Service (business logic) layer: the flow is **Controller** → **Service** → **Repository**, logic lives in the service.
- **DTOs** vs entities — never expose entities directly; use **TicketDto**, **CreateTicketDto**, **UpdateTicketDto**.
- **AutoMapper**: mapping entity ↔ DTO with a profile instead of copying properties by hand.

Resources

- Business logic layer: dotnettutorials.net/lesson/business-logic-layer-in-asp-net-core
- DTOs in Web API: dotnettutorials.net/lesson/dto-in-asp-net-core-web-api
- AutoMapper getting started: docs.automapper.org/en/stable/Getting-started.html

Practice

- Create **ITicketService** + **TicketService** in **.BL**; define the ticket DTOs in **.Shared**.
- Add an AutoMapper profile and register it in DI; map **Ticket** ↔ **TicketDto**.
- The service uses the repository and returns **DTOs** (not entities).

✓ **End of day: the service returns mapped DTOs for tickets, with AutoMapper doing the conversion.**

Day 4 Task: Full CRUD Endpoints (Thin Controller)

Learn

- The **thin controller** principle: no logic in the controller — it only depends on **ITicketService**.
- The five actions: **GET** all, **GET** /{id}, **POST**, **PUT** /{id}, **DELETE** /{id}.
- Proper responses and **async/await**: **Ok**, **NotFound**, **CreatedAtAction**, **NoContent**.

Resources

- Controller actions & return types: learn.microsoft.com/aspnet/core/web-api/action-return-types
- Attribute routing: learn.microsoft.com/aspnet/core/mvc/controllers/routing
- Async programming in C#: learn.microsoft.com/dotnet/csharp/asynchronous-programming

Practice

- Build **TicketsController** with all five CRUD actions calling the service.
- Test each one in Swagger against the seeded database; confirm a created ticket lands in the DB.
- Keep the controller thin — if logic creeps in, push it down into the service.

✓ **End of day: full CRUD for tickets works end-to-end through the API, Service and Repository layers.**

Day 5 Task: API Versioning + Verify + Final PR

Learn

- Why API versioning matters, and the URL-segment style: **/api/v1/tickets**.
- Adding the versioning package (**Asp.Versioning.Mvc**) and tagging controllers with a version.
- A final walk through the full Clean Architecture flow, layer by layer.

Resources

- API versioning (guide): dotnettutorials.net/lesson/web-api-versioning
- Asp.Versioning docs: github.com/dotnet/aspnet-api-versioning/wiki
- Full Web API walkthrough (CRUD recap): learn.microsoft.com/aspnet/core/tutorials/first-web-api

Practice

- Add versioning so the endpoints live under **/api/v1/tickets**.
- Re-test the full CRUD on the versioned routes in Swagger.
- Tidy up, commit, and open the final Pull Request of the week (branch → commit → push → PR).

✓ **End of day: a versioned, full-CRUD TicketDesk API on Clean Architecture, verified end-to-end and pushed via a clean PR.**

💡 **Tip:** keep each task on its own branch + PR (e.g. **feature/repository**, **feature/ticket-crud**). The days are ordered on purpose — Day 4 needs Day 3, which needs Day 2 — so review each PR before the next task starts.

Notes for the Trainer

- This is the **heaviest week so far** — it merges two phases (API foundation + full CRUD). If the trainee is slower, let versioning (Day 5) slip to early next week rather than skipping the patterns.
- Guard the **Clean Architecture direction**: `.Domain` references nothing; EF Core stays in `.DAL`; no `DbContext` in controllers.
- Insist on **thin controllers** — the moment business logic appears in a controller, move it to the service.
- DTOs are mandatory: entities must never be returned directly from an endpoint.
- Every endpoint must work against the **real Week 3 database**, not fake in-memory data.
- Git stays mandatory: every task goes through a branch + PR, reviewed in order.
- React is parked — do not let it distract from getting the backend patterns right first.
- If the trainee finishes early: add 2–3 LeetCode Easy problems (not a priority).